

Reinforcement Learning

Chapter 3: Model-free RL

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Fall 2026

Terminating Monte-Carlo: *Sample Inefficiency*

! Monte-Carlo is *sample inefficient*

- We are *loosing* lots of states in each *sample trajectory*!

A better solution is to use the *recursive property* of return and do

bootstrapping

To understand this idea, let's think a bit science-fictional 😊

Genie in Box

Assume a *genie* can tell us the value $v_{\pi}(s)$ for *each state* s . We want to test this *genie* via a numerical algorithm.

Testing Genie in the Box

Bellman tells us that at any *state* s , we should see

$$v_{\pi}(s) = \mathbb{E} \{R_{t+1} | S_t = s\} + \gamma \mathbb{E} \{v_{\pi}(S_{t+1}) | S_t = s\}$$

Monte-Carlo tells us further that after K *sample trajectories*

$$S_0, A_0 \xrightarrow{R_1} S_1$$

initiated at $S_0 = s$ and *terminated after only one step*, we have

$$\mathbb{E} \{R_{t+1} | S_t = s\} \approx \frac{1}{K} \sum_{k=1}^K R_1[k]$$

$$\mathbb{E} \{v_{\pi}(S_{t+1}) | S_t = s\} \approx \frac{1}{K} \sum_{k=1}^K v_{\pi}(S_1[k])$$

Testing Genie in the Box

We could hence find an estimator of $v_\pi(s)$ as

$$v_\pi(s) \approx \hat{v}_\pi(s) = \frac{1}{K} \sum_{k=1}^K R_1[k] + \gamma v_\pi(S_1[k])$$

And, of course we could simply evaluate this estimator **online** as

$$\hat{v}_\pi(s) \leftarrow \hat{v}_\pi(s) + \frac{1}{K} (R_1 + \gamma v_\pi(S_1) - \hat{v}_\pi(s))$$

if we need linear averaging or alternatively as

$$\hat{v}_\pi(s) \leftarrow \hat{v}_\pi(s) + \alpha (R_1 + \gamma v_\pi(S_1) - \hat{v}_\pi(s))$$

if we think of more general weighted averaging

Computing Values via Bootstrapping: Algorithm 1

We can write our genie-testing algorithm as

$\text{TD_verI}(\pi, s):$

- 1: Initialize estimator of value as $\hat{v}_{\pi}(s) = 0$
- 2: Ask genie $v_{\pi}(\bar{s})$ for all $\bar{s}$ that can be followed after s
- 3: **for** episode = 1 : K **do**
- 4: Initialize with **state** $S_0 = s$ and act via policy $\pi(a|s)$
- 5: Sample a single-step **terminated** trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1$$

- 6: Update estimate of value as $\hat{v}_{\pi}(s) \leftarrow \hat{v}_{\pi}(s) + \alpha(R_1 + v_{\pi}(S_1) - \hat{v}_{\pi}(s))$
- 7: **end for**

Note that in this algorithm we **don't** need the environment to be **episodic**, as we use **recursive property** of value-function!

This explains the idea of **bootstrapping**

Bootstrapping: Using Value Estimates

- + But, in practice we don't have *genie*!
- Absolutely! But, we may use *this property*

We can replace those true values with their estimates

- They are initially *bad* estimates
 - ↳ and thus return in bad estimate for *the other state*
- They *gradually improve*
 - ↳ and therefore return *better estimate* for *the other state*

The key point is that we get rid of the necessity of having a *terminal state*!

Computing Values via Bootstrapping: Algorithm II

So, we could get rid of the *genie* finally

$\text{TD_verII}(\pi, s)$:

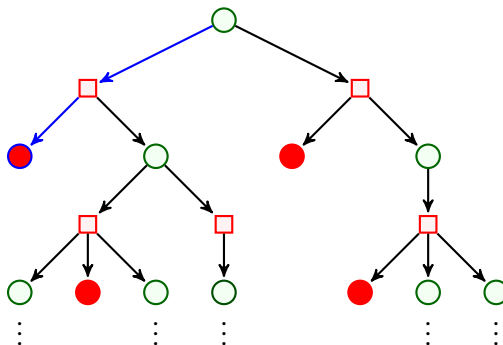
- 1: Initialize estimator of value as $\hat{v}_{\pi}(s) = 0$
- 2: Use available $\hat{v}_{\pi}(\bar{s})$ for all $\bar{s}$ that follow s
- 3: **for** episode = 1 : K **do**
- 4: Initialize with *state* $S_0 = s$ and act via policy $\pi(a|s)$
- 5: Sample a single-step *terminated* trajectory

$$S_0, A_0 \xrightarrow{R_1} S_1$$

- 6: Update estimate of value as $\hat{v}_{\pi}(s) \leftarrow \hat{v}_{\pi}(s) + \alpha(R_1 + \hat{v}_{\pi}(S_1) - \hat{v}_{\pi}(s))$
- 7: **end for**

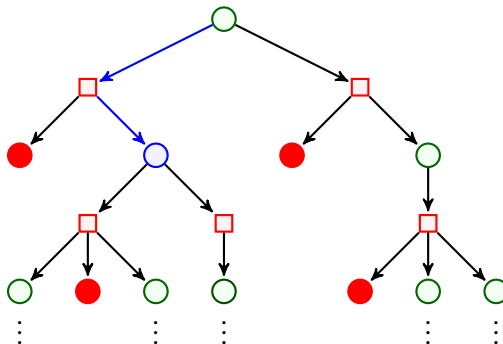
Backup Diagram: *Sampling with Bootstrapping*

Looking at the backup tree, we are now sampling *one-step trajectories*



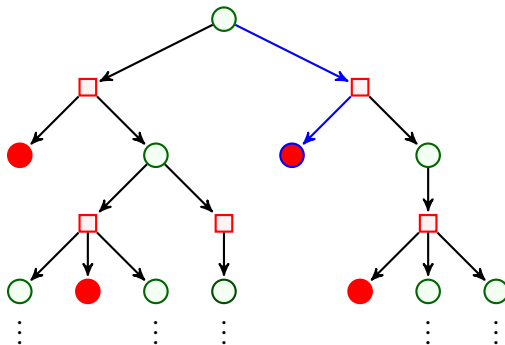
Backup Diagram: *Sampling with Bootstrapping*

Looking at the backup tree, we are now sampling *one-step trajectories*



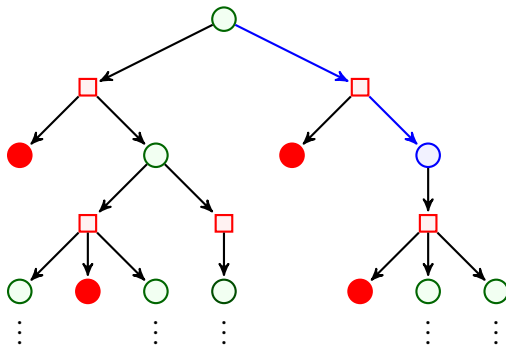
Backup Diagram: *Sampling with Bootstrapping*

Looking at the backup tree, we are now sampling *one-step trajectories*



Backup Diagram: *Sampling with Bootstrapping*

Looking at the backup tree, we are now sampling *one-step trajectories*



Temporal Difference

Bootstrapping can replace **Monte-Carlo** in our evaluation algorithms

- 1 We start with some initial value estimates
- 2 We sample a trajectory of finite length T

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

- ↳ it can either end with a terminal state if we have any
- ↳ or could simply be terminated

- 3 We move over trajectory and update value of each state by **bootstrapping**

This idea of estimating values is called

Temporal Difference $\equiv$ TD

Temporal Difference: TD-0

TD_Eval(π):

- 1: Initialize estimator of value as $\hat{v}_{\pi}(s^n) = 0$ for $n = 1 : N$
- 2: **for** episode = 1 : K **do**
- 3: Initialize with a **random state** S_0 and act via policy $\pi(a|s)$
- 4: Sample a trajectory until either a **terminal** state or some **terminating** T

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

- 5: **for** $t = 0 : T - 1$ **do**
- 6: Update as $\hat{v}_{\pi}(S_t) \leftarrow \hat{v}_{\pi}(S_t) + \alpha(R_{t+1} + \gamma\hat{v}_{\pi}(S_{t+1}) - \hat{v}_{\pi}(S_t))$
- 7: **end for**
- 8: **end for**

Attention

With TD, we even don't need to wait till a trajectory is sampled!

Evaluating Action-Values via Bootstrapping

- + What about the **action-values**? Can we **bootstrap** again?
- Sure!

Recall **Bellman equation** of action-value function

$$q_{\pi}(s, a) = \mathbb{E} \{R_{t+1} | S_t = s, A_t = a\} + \gamma \mathbb{E} \{v_{\pi}(S_{t+1}) | S_t = s, A_t = a\}$$

So, we can use sample trajectory to estimate **action-values** as well: at time t , we can update estimate of pair (S_t, A_t) as

$$\hat{q}_{\pi}(S_t, A_t) \leftarrow \hat{q}_{\pi}(S_t, A_t) + \alpha(R_{t+1} + \gamma \hat{v}_{\pi}(S_{t+1}) - \hat{q}_{\pi}(S_t, A_t))$$

Temporal Difference: Action-Value

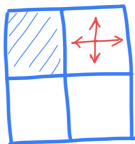
TD_QEval(π):

- 1: Initialize estimator of value as $\hat{q}_\pi(s^n, a^m) = 0$ for $n = 1 : N$ and $m = 1 : M$
- 2: **for** episode = 1 : K **do**
- 3: Initialize with a **random state-action pair** (S_0, A_0) and act via policy $\pi(a|s)$
- 4: Sample a trajectory until either a **terminal** state or some **terminating** T

$$S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \dots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

- 5: **for** $t = 0 : T - 1$ **do**
- 6: Set $\hat{q}_\pi(S_t, A_t) \leftarrow \hat{q}_\pi(S_t, A_t) + \alpha(R_{t+1} + \gamma \hat{v}_\pi(S_{t+1}) - \hat{q}_\pi(S_t, A_t))$
- 7: **end for**
- 8: **end for**

Example: Dummy Grid World with Random Walk



Let's get back to our *dummy world*: we now use *TD* to compute the values for *uniform random policy*, i.e.,

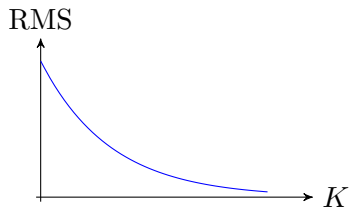
$$\pi(a|s) = \frac{1}{4}$$

for all *actions* and *states*

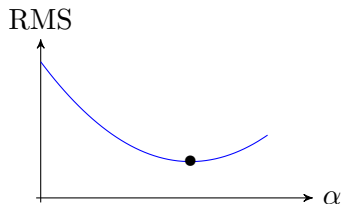
Try it at home 😊

Typical Behavior: *Sample Size and Update Rate*

We see the same behavior also with TD: *against K* we see



and against *the update rate α* we have a *minimum*



Example: TD vs Monte-Carlo in Single-Button Game



We got a single button to push $\rightsquigarrow$ each time we push

- we either get into **green** or **blue** mode allowing us *to push the button again* and returns a **0/1** reward
- or it gets into **red** mode which *only returns a 0/1 reward* and **game is over**

Obviously this game has

- Three states: **green**, **blue** and **red** which is **terminal**
- a single action, i.e., pushing the button

Example: TD vs Monte-Carlo in Single-Button Game



We play this game 6 episodes and get following *sample trajectories*

Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red

Blue $\xrightarrow{1}$ red

Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red

Blue $\xrightarrow{1}$ red

Blue $\xrightarrow{0}$ red

Green $\xrightarrow{0}$ Blue $\xrightarrow{0}$ red

Let's estimate $v(\text{blue})$ and $v(\text{green})$ by both TD-0 and Monte-Carlo

Example: TD vs Monte-Carlo in Single-Button Game

Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{0}$ red
 Green $\xrightarrow{0}$ Blue $\xrightarrow{0}$ red

With Monte-Carlo approach, we could say: we have 8 sample trajectories starting with **Blue** with 4 returning 1 and 4 returning 0; thus we have

$$\hat{v}(\text{blue}) = \frac{4}{8} = 0.5$$

We also have only one sample trajectory starting at **Green** with zero return; thus, we have

$$\hat{v}(\text{green}) = \frac{0}{1} = 0$$

Example: TD vs Monte-Carlo in Single-Button Game

Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{0}$ red
 Green $\xrightarrow{0}$ Blue $\xrightarrow{0}$ red

With TD-0: *before getting to Green* we have 7 Blue states followed by either Blue or red $\rightsquigarrow$ let's say

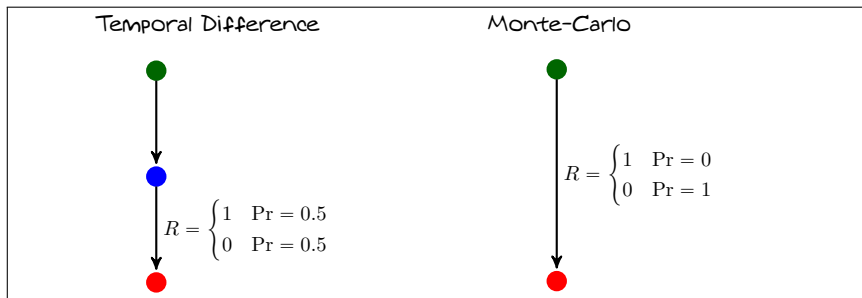
$$\hat{v}(\text{Blue}) \approx 0.5$$

We now get to *Green* $\rightsquigarrow$ we have not seen it before, thus we update as

$$\hat{v}(\text{Green}) \leftarrow \underbrace{\hat{v}(\text{Green})}_0 + \left(\underbrace{R_{t+1}}_0 + \underbrace{\hat{v}(S_{t+1})}_{\hat{v}(\text{Blue}) \approx 0.5} - \underbrace{\hat{v}(\text{Green})}_0 \right) \approx 0.5$$

Temporal Difference vs Monte-Carlo: Note I

This is a **fundamental difference**: in **Monte-Carlo** we only look at best approximation given data, without looking into the **Markovity** of the state, whereas in **TD** we take into account the fact that we are dealing with an **MDP**



Temporal Difference vs Monte-Carlo: Note I



This is common to see in the literature that people say

- TD finds **maximum-likelihood** estimate of the values
 - ↳ It uses this assumption that the state is a **Markov process**
 - ↳ It's the **better option**, if we have access to the **complete (Markov) state**
- Monte-Carlo finds **least-squares** estimate of the values
 - ↳ It **ignores Markovity** of the state
 - ↳ Maybe **better option**, when we **cannot** access the **complete (Markov) state**

Back to Single-Button Game: *Side Note on Batch Updating*



What would happen, if we get *sample trajectories* in the following order

Green $\xrightarrow{0}$ Blue $\xrightarrow{0}$ red

Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red

Blue $\xrightarrow{1}$ red

Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red

Blue $\xrightarrow{1}$ red

Blue $\xrightarrow{0}$ red

Back to Single-Button Game: *Side Note on Batch Updating*

green $\xrightarrow{0}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{0}$ red

If we go *only once* over the *batch* of all episodes with TD, we get

$$\hat{v}(\text{green}) \approx 0$$

$$\hat{v}(\text{blue}) \approx 0.5$$

Back to Single-Button Game: *Side Note on Batch Updating*

green $\xrightarrow{0}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ Blue $\xrightarrow{0}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{0}$ Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{1}$ red
 Blue $\xrightarrow{0}$ red

If we go *twice* over the *batch* of all episodes with TD, we get

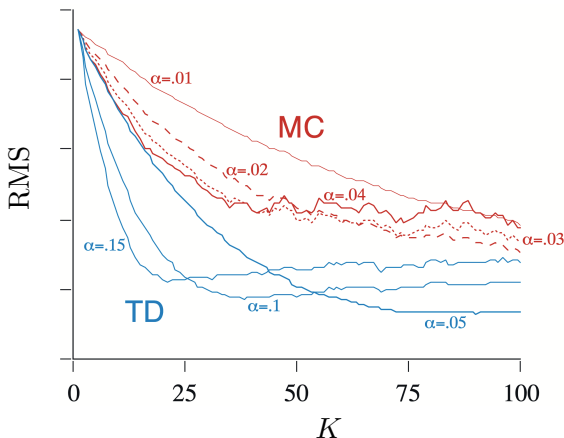
$$\hat{v}(\text{green}) \approx 0.5$$

$$\hat{v}(\text{blue}) \approx 0.5$$

We get *better* if we go over the *batch of data* multiple times!

Temporal Difference vs Monte-Carlo: Note II

Let's see how Monte-Carlo performs against TD-0 algorithm for a bit larger example of *random walk* on a grid¹



¹This figure is taken from Chapter 6 of Sutton and Barto's book

Temporal Difference vs Monte-Carlo: Note II

What has been seen in the diagram is a **general behavior**

- Monte-Carlo is good in sense of **bias** but bad in terms of **variance**

↳ It always returns an unbiased estimator of **value**, i.e.,

$$\mathbb{E} \{ \hat{v}_{\pi}(s) \} = v_{\pi}(s)$$

↳ It's estimation is however high **variance**, i.e.,

$$\mathbb{E} \left\{ (\hat{v}_{\pi}(s) - v_{\pi}(s))^2 \right\} \rightsquigarrow \text{large}$$

- TD-0 is good in sense of **variance** but can be bad in terms of **bias**

↳ It can return a biased estimator of **value**, i.e.,

$$\mathbb{E} \{ \hat{v}_{\pi}(s) \} \neq v_{\pi}(s)$$

↳ It's estimation is low **variance**, i.e.,

$$\mathbb{E} \left\{ (\hat{v}_{\pi}(s) - v_{\pi}(s))^2 \right\} \rightsquigarrow \text{small}$$

Policy Iteration with TD-0

We can use TD-0 to implement another variant of GPI

TD_PolicyItr():

- 1: Initiate two random policies π and $\bar{\pi}$
- 2: **while** $\pi \neq \bar{\pi}$ **do**
- 3: $\hat{q}_\pi = \text{TD_QEval}(\pi)$ and $\pi \leftarrow \bar{\pi}$
- 4: $\bar{\pi} = \text{Greedy}(\hat{q}_\pi)$
- 5: **end while**

And, recall that our greedy algorithm is

Greedy(q_π):

- 1: **for** $n = 1 : N$ **do**
- 2: Improve the policy by taking *deterministically* the *best action*

$$\bar{\pi}(a^m | s^n) = \begin{cases} 1 & m = \underset{m}{\operatorname{argmax}} q_\pi(s^n, a^m) \\ 0 & m \neq \underset{m}{\operatorname{argmax}} q_\pi(s^n, a^m) \end{cases}$$

- 3: **end for**

GPI with TD-0: Visualization

We can plot the same figure again in this case

